



Lecture[1]

C FUNDAMENTALS



Agenda

- Data type and storage
- Expression
- Control flow



Agenda

- Data type and storage
- Expression
- Control flow



Data Type

- C has the following simple data types:

Data type	C code	Size in bytes	Range
Char or signed char	char	1	-128 to 127
Unsigned char	unsigned char	1	0 to 255
int or signed int	int	at least 2	-32768 to 32767
Unsigned int	unsigned int	at least 2	0 to 65535
Short int or signed short int	short	2	-32768 to 32767
Unsigned short int	unsigned short	2	0 to 65535
Long int or Signed long int	long	at least 4	-2147483648 to 2147483647
Unsigned long int	unsigned long	at least 4	0 to 4294967295
float	float	4	3.4E-38 to 3.4E+38
double	double	8	1.7E-308 to 1.7E+308
Long double	long double	10*	3.4E-4932 to 1.1E+4932

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6
7      int b=3;
8      //print out the value of b
9      printf("the size of b is %u", sizeof(b));
10     return 0;
11 }
```

sizeof 是一个编译时运算符而非函数，它的核心作用是在编译阶段就确定其操作数（可以是数据类型或具体变量）所占用的内存字节数。

the size of b is 4

On 32-bit machine is usually 4 bytes (32bits)



- **#include**是一个预处理指令，其核心作用是在编译代码之前，将指定文件（通常是头文件）的全部内容原封不动地“复制粘贴”到当前指令所在的位置；这使我们能够使用标准库函数（如 printf 需要通过 #include <stdio.h> 引入其声明）或共享自定义的代码模块（使用 #include "myheader.h"），从而极大地提高了代码的复用性和项目结构的清晰度。
- **stdio.h**是标准输入输出（Standard Input & Output）的头文件，它提供了与外部设备进行数据交换的核心函数，使得程序能够从键盘读取输入（如使用 scanf）、向屏幕输出结果（如使用 printf）、以及对文件进行读写操作（如使用 fopen 和 fclose），是实现用户交互和数据持久化最基本且必不可少的工具集。
- **stdlib.h**是标准库（Standard Library）的头文件，它提供了大量通用且至关重要的函数，包括内存管理（如 malloc 和 free 用于动态分配和释放内存）、随机数生成（rand）、字符串与数字的转换（atoi）、进程控制（如 exit 用于终止程序）、以及环境交互（如 system 执行系统命令）等功能，是进行除基础输入输出外各种系统级和通用编程操作的核心工具集。



Unsigned int

- How to store an unsigned int
 - All bits are used to store value

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$1111_2 = 15_{10}$$

- 2 Bytes = 16 bits
- $0 \sim 2^{16} - 1 = 65535$



int with sign

- Need to both represent positive and negative int
- Sign and magnitude (原码)
 - Use highest bit for sign: 0 -> +, 1 -> -
 - Other bits are used to store value

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$1111_2 = 15_{10}$$

– But

- What about 0? Two representations
- $15 + (-15) = ?$ $00001111 + 10001111 = ?$

1	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$1000111_2 = -15_{10}$$



One's-Complement for signed numbers

- One's-Complement (二进制反码)

- Use highest bit for sign: 0 -> +, 1 -> -
- Positive number X is encode of X
- Negative number X is encode $\sim X$ and highest bit is 1

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

15_{10}

1	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

-15_{10}

– But

- What about 0? Two representations
- $15 + (-15) = ?$ $00001111 + 11110000 = ?$



Two's-Complement for signed numbers

- Two's-Complement (二进制补码)
 - Use highest bit for sign: 0 -> +, 1 -> -
 - Positive number X is encode of X
 - Negative number X is encode of $2^n + X$ and highest bit is 1

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

15_{10}

1	1	1	1	0	0	0	1
---	---	---	---	---	---	---	---

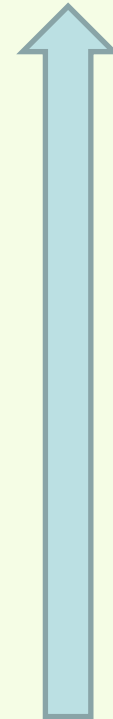
-15_{10}

(方法2: 反码+1)



Two's-Complement for signed numbers

- Only one 0
- $-2^{n-1} \sim 2^{n-1}-1$
- The order is respected
- Arithmetic calculation



Two's complement	Decimal
[0]111	+7
[0]110	+6
.....	
[0]001	+1
[0]000	0
[1]111	-1
.....	
[1]001	-7
[1]000	-8



- 原码、反码和补码是计算机用来表示有符号整数的三种编码方式;
- **原码**最直观, 最高位表示符号 (0正1负), 其余位表示绝对值, 但会导致“零”有两种表示形式 (+0 和 -0) 且加减运算复杂;
- **反码**是在原码基础上对负数各位取反得来, 虽然简化了运算但“零”的问题依然存在;
- **补码**则完美解决了这些问题, 它是在反码基础上加1得到, 使得加减运算可以统一用加法器完成, “零”只有唯一表示, 并且能多表示一个最小值, 因此成为了现代计算机中有符号整数存储的实际标准方案。
- 对于正数: 原码 = 补码 = 反码 (三者相同)
- 对于负数: 原码 $\rightarrow$ 补码: 符号位不变, 数值位取反+1
补码 $\rightarrow$ 原码: 符号位不变, 数值位取反+1



Floating types

float

single-precision floating-point

double

double-precision floating-point

long double

extended-precision floating-point

/	Smallest Positive Value	Largest Value	Precision	Size
float	1.17×10^{-38}	3.40×10^{38}	7 digits	4 Bytes
double	2.22×10^{-308}	1.79×10^{308}	15 digits	8 Bytes

float x;

scanf("%f", &x);

printf("%f", x);

double x;

scanf("%lf", &x);

printf("%f", x);

long double x;

scanf("%Lf", &x);

printf("%Lf", x);



How to represent a float

- How to represent a decimal fraction in binary?

– $10.125 \rightarrow 1010.001_2$

- Formatted form

– $1010.001_2 = 0.1010001_2 \times 2^{100}_2$

- Encoding of float

– $N = (-1)^S \times 0.M \times 2^E$

$$\begin{aligned} 0.125 * 2 &= 0.25 \\ 0.25 * 2 &= 0.5 \\ 0.5 * 2 &= 1.0 \end{aligned}$$

一共将小数点**向左**
移动了4位，4的二
进制是100

S	E	M
0	000100	001010001

10.125 in 16 bits



IEEE 754 float (4 Bytes)

符号位 (S)

指数位 (E)

尾数位 (M)



1 bit

8 bits

23bits

- M is consider as 0.1xxxx->1.xxxx
– Save one bit

- E use $E = E_{\text{real}} + 127$

- $N = (-1)^S \times (1 + M) \times 2^{E-127}$

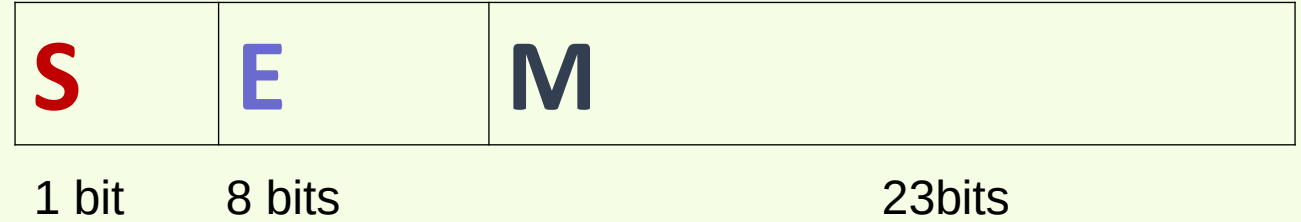
M是23位尾数位表示的二进制小数。这里有一个隐藏的**前导1**，所以实际的值是1.xxx。因为这个前导的1总是存在，所以为了节省一位宝贵的存储空间，在将数字存入内存时，这个1被省略（隐含）了。在从内存中读取数字进行计算时，这个1会被自动加回去。

这里的 127 称为**指数偏移 (Exponent Bias)**。8位指数本身是无符号数（范围 0-255），减去127后，实际指数范围是 **-126 到 +127**。



IEEE 754 float (4 Bytes)

- $N = (-1)^S \times (1+M) \times 2^{E-127}$
- The range is



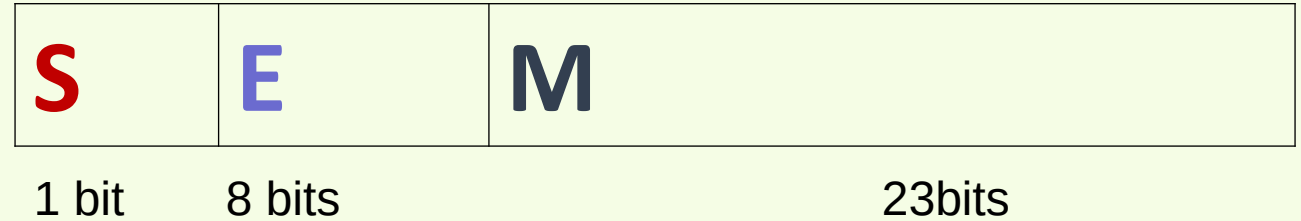
– Smallest positive value: (规约数, IEEE 754标准)

- 符号位 $S = 0$ (正数)
- 指数位 E 尽可能小: E 不能全为0 (全为0表示次正规数), 所以最小为00000001 (1) 。
- 尾数位 M 尽可能小: 23位全部为 0, $M=1.00...0$
- $N_{\min} = 1.0 \times 2^{1-127} = 2^{-126} \approx 1.17549435 \times 10^{-38}$



IEEE 754 float (4 Bytes)

- $N = (-1)^S \times (1+M) \times 2^{E-127}$
- The range is

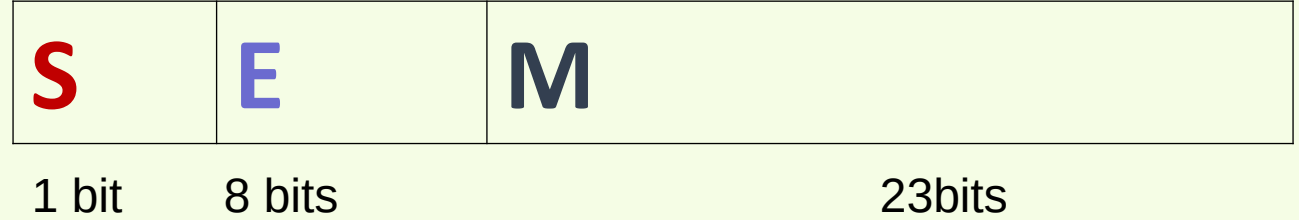


– Biggest positive value:

- 符号位 $S = 0$ (正数)
- 指数位 E 尽可能大: E 不能全为1 (全为1表示无穷大) , 所以最大为 11111110 (254) 。
- 尾数位 M 尽可能大: 23位全部为 1, $M=1.11...1$
- $N_{\max} = 1.11...1 \times 2^{254-127} = (2-2^{-23}) \times 2^{127} \approx 3.4028235 \times 10^{38}$



IEEE 754 float (4 Bytes)



– Precision:

- float 使用 23 位来存储尾数，加上隐含的 “前导1”，实际有 24 位的有效二进制精度。
- 计算能表示的十进制精度可以通过公式 $\log_{10}(2^n)$ 来估算，其中 n 是二进制位数。
 $\log_{10}(2^{24}) \approx \log_{10}(16,777,216) \approx 7.225$
- 这意味着 24 位二进制精度大约相当于 7.225 位十进制精度。因此，我们保守地说，精度是 7 位十进制数字。
- 这表示你对一个 float 变量赋值时，无论你输入的数字有多长，计算机最多只能安全地存储大约前7位有效数字。第8位有效数字开始 “失真” 。



IEEE 754 float (4 Bytes)

例 1：存储一个7位数字

```
float f = 123.4567; // 7位有效数字
```

这个值很可能被精确存储，没有误差。

例 2：存储一个超过7位的数字

```
float f = 123.456789; // 9位有效数字
```

```
printf("%.9f\n", f); // 输出可能类似：123.456787109
```

你希望存储 123.456789，但 float 的精度只有 ~7 位。计算机会将这个数字舍入到它能表示的最近似的值。当你打印出来时，你可能会看到

123.456787109。注意：前7位数字 (123.4567) 很可能是正确的。从第8位开始 (89 vs 87109)，后面的数字就失真了，它们是舍入误差的结果，不再可靠。



IEEE 754 float (4 Bytes)

例 3：大数的问题（更明显）

```
float f = 123456789.0f; // 9位有效数字 (123456789)
```

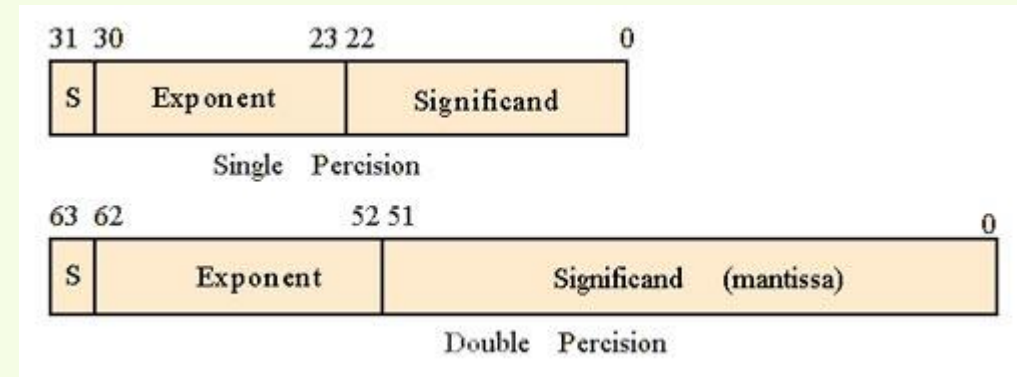
```
printf("%f\n", f); // 输出很可能为：123456792.000000
```

你希望存储 123,456,789。但 float 的精度只有7位有效数字，意味着它无法精确区分千万级别数字的个位和十位。它只能将这个大数舍入到最接近的、它能表示的值，结果是 123,456,792。你会发现，前7位数字 1234567 是稳定的，但后面的 89 变成了 92，已经不再准确。



Float/double precision

- Float and double are not precise
 - The storage is not accurate.
 - Floating operation uses specific hardware
- Use double if no specific requirement





Exe

- What's the height of LBJ
- How 6 feet 9 inches 2.06m?
- **Meter = (foot +inch/12)*0.3048**
- Write a program

```
int main()
{
    printf("input foot and inch:\n");
    int foot;
    int inch;
    ...
    return 0;
}
```



Los Angeles Lakers | #6 | Forward

LeBron James

PPG	RPG	APG	PIE	HEIGHT	WEIGHT
25.0	7.7	7.8	19.1	6'9" (2.06m)	250lb (113kg)
				AGE	BIRTHDATE
				36 years	December 30, 1984



Character Types

- Char is also an integer type

- 'a', '1'

- %c in scanf and printf

- ASCII code

- '1'的ASCII编码是49,
 - 当变量a==49, a的值就是'1'

```
char ch;  
int i;  
i = 'a';           // i is now 'a', its value is 97  
ch = 65;           // ch is now 'A' , its value is 65  
ch = ch + 1;       // ch is now 'B', its value is 66  
ch++;              //ch is now 'C', its value is 67
```

```
for(ch = 'A'; ch <= 'Z'; ch++)  
{  
    printf("%c\n",ch);  
}
```



Boolean

- Boolean: {true, false}
- There is no Boolean data type in C
- Any integer $\neq 0$ is considered true
- 0 is considered false

```
int a= 3;
```

```
int b = !a; // b=0
```



Constants int

- Integer constants: (整型常量)

- Decimal: 11,12345

- Declare a named constant:

- Using **const** keyword

```
const int PAIR=2
```

- Using **#define** pre-processor directive

```
#define PAIR 2
```

预处理指令，在编译之前，把所有PAIR简单替换成 2。

```
int main()
{
    int n;
    const int TIMES = 2;
    scanf("%d",&n);
    printf("Output is %d\n",TIMES*n);
}
```



Constants

- Character constants
- String constants
 - "I am a string"
 - Character array
 - always null ('\0') terminated.
 - (see <string.h> for string functions)



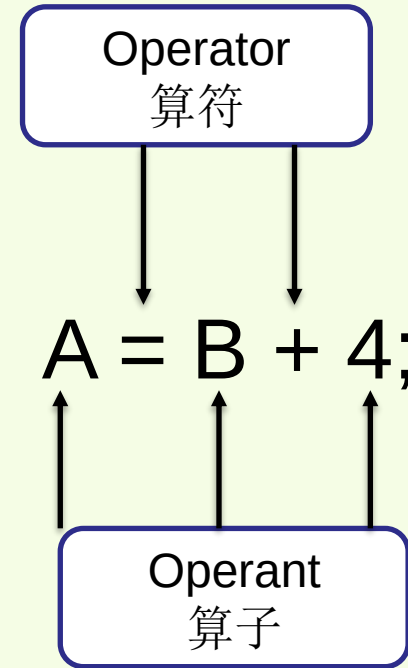
Agenda

- Data type and storage
- Expression
- Control flow



Expressions

- Arithmetic operator:
– $+, -, *, /, \%$
- Relation operators:
– $<, >, <=, >=, ==, !=$
- Logical operator:
– $\&\&$ (and), $||$ (or)
- Increment and decrement operators:
– $++, --, ++, --$
- Assignment operators: $+=, -=, /= \dots$





Relation Operators

- Relation operators has lower precedence than arithmetic operators:

—>, >=, <, <=

—==, !=

- Don't confuse = and == !

```
int x=5;
if (x==6)    /* false */
{
    /* ... */
}
/* x is still 5 */
```

```
int x=5;
if (x=6)    /* always true */
{
    /* x is now 6 */
}
/* ... */
```



Increment and Decrement Operators

<code>x++</code>	post-increment <code>x</code>	<code>++x</code>	pre-increment <code>x</code>
<code>x--</code>	post-decrement <code>x</code>	<code>--x</code>	pre-decrement <code>x</code>

Note the difference between `++x` and `x++`:

```
int x=5;
int y;
y = ++x;
/* x == 6, y == 6
*/
```

```
int x=5;
int y;
y = x++;
/* x == 6, y == 5
*/
```



Assignment Operators

- Most binary operators have a corresponding assignment operator *op*
 - $+, -, *, /, \%, <<, >>, \&, ^, |$
 - $expr1\ op =\ expr2 \iff expr1 = (expr1)\ op\ (expr2)$
 - $x\ *=\ y+1 \iff x = x * (y+1)$
- Assignment statement has a value
- 赋值语句本身也是一个表达式，这个表达式会产生一个值。这个值就是被赋给变量的那个值。
- 赋值语句 $x = 5$ 不仅仅让 x 变成 5，它本身也具有值，这个值就是 5。



Assignment Operators

例1：链式赋值

```
int a, b, c;
```

```
a = b = c = 100; // 链式赋值
```

- 执行顺序和含义：
 - $c = 100$: 把 100 赋给 c 。这个表达式本身的值是 100。
 - $b = (c = 100)$: 等价于 $b = 100$ 。把上一步的结果 100 赋给 b 。这个表达式本身的值也是 100。
 - $a = (b = (c = 100))$: 等价于 $a = 100$ 。把上一步的结果 100 赋给 a 。
- 最终, a, b, c 三个变量的值都变成了 100。



Assignment Operators

例2：在循环条件中赋值

```
int value;  
while ((value = get_next_value()) != -1) { // 在条件中赋值并判断  
    printf("%d\n", value);  
}
```

- (value = get_next_value()) 这个赋值语句的值就是从函数 get_next_value() 获取到的值。
- 然后立即用 != -1 来判断这个值是否为结束标志。
- 这样写非常紧凑，避免了先调用函数、再判断、再循环的繁琐写法。



Bitwise Operations

- Applied to char, int, short, long
 - And &
 - Or |
 - one's complement ~
 - Exclusive Or ^
 - Left-shift <<
 - Right-shift >>



Bitwise Operations

0x 16进制

Operator	Typical Usage	Express
& (AND)	1. Take one specific bit	$n \& 0x10$; (5 th bit)
	2. Put several bits to 0 while keep others	$n = n \& 0xfffff00$; (lower 8 bits to 0)
(OR)	1. Put several bits to 1 while keep others	$n = n 0xff$; (lower 8 bits to 1)
~ (Complement)	1. Put all bits to complement (not “+ to -”)	$n = \sim n$; ($\sim 1 \rightarrow -2$)
^ (XOR)	1. Put one bit or a set of bits to complement	$n = n \wedge 0x10$; (5 th bit turns to its complement)
	2. Switch two values	$a = a \wedge b$; $b = a \wedge b$; $a = a \wedge b$;
<<	1. Equivalent to $*2^i$	$n \ll i$;
>>	2. Equivalent to $/2^i$	$n \gg i$;



Example: Bit Count

```
/*  
    count the 1 bits in a number  
    e.g. bitcount(0x45) (01000101 binary) returns 3  
*/  
  
int bitcount (unsigned int x) {  
    int b;  
  
    for (b=0; x != 0; x = x >> 1)  
        if (x & 01) /* octal 1 = 000000001 */  
            b++;  
  
    return b;  
}
```



Operator Precedence and Associativity

highest: + - (unary)

* / %

lowest: + - (binary)

$$-i * -j = (-i) * (-j)$$

$$+i + j / k = (+i) + (j / k)$$

left associative: it groups from left to right

right associative: it groups from right to left

The binary arithmetic operators (*, /, %, + and -) are all left associative (from left to right)

$$i - j - k = (i - j) - k \quad i * j / k = (i * j) / k$$

The unary arithmetic operators(+ and -) are both right associative

$$- + i = - (+i)$$



Expression Evaluation

Precedence	Name	Symbol(s)	Associativity 结合关系
1	X++/X--		Left (from left to right)
2	++X/--X unary +/- (单目)		Right (from right to left)
3	multiplicative	*, /, %	left
4	additive	+, -	left
5	logical	&&,	left
6	assignment	=, *=, /=, +=, -=	Right



Expression Evaluation examples (1/2)

```
int a,b,c,d,e,f;
```

```
a=1;   b=2;   c=3;
```

```
d = !(a+b)+c-1&&b+c/2;
```

```
e = a--&&b++&&++c;
```

```
f = a+=b++-c*d ;
```

What are the values of a,b,c,d,e,f?



Expression Evaluation examples(2/2)

```
int a,b,c,d,e,f;
```

```
a=1;  b=2;  c=3;
```

```
d = c-1&b;
```

```
e = a--&b;
```

```
f = a|=b++ ;
```

What are the values of a,b,c,d,e,f?



Conditional Expressions

- Conditional expressions
- `expr1? expr2:expr3;`
- if `expr1` is true then `expr2` else `expr3`

```
(u%2==0)?printf("even num\n"):printf("odd num\n");
```



Agenda

- Data type and storage
- Expression
- Control flow



Control Structures

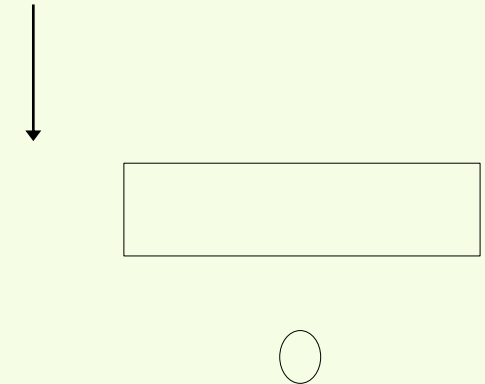
- Sequence execution
 - Statements executed one after the other in the order written
- Selection structures:
 - if, if/else, and switch
- Repetition (Loops) structures:
 - while, for and do/while
- Transfer controls:
 - goto, breaks, continues



Control Structures

- Flowchart

- Graphical representation of an algorithm
- Drawn using certain special-purpose symbols connected by arrows called *flowlines*.
- Rectangle symbol (action symbol): indicates any type of action.
- Oval symbol: indicates beginning or end of a program, or a section of code (circles).



- Single-entry/single-exit control structures

- Connect exit point of one control structure to entry point of the next (*control-structure stacking*).
- Makes programs easy to build



The `if` Selection Structure (I)

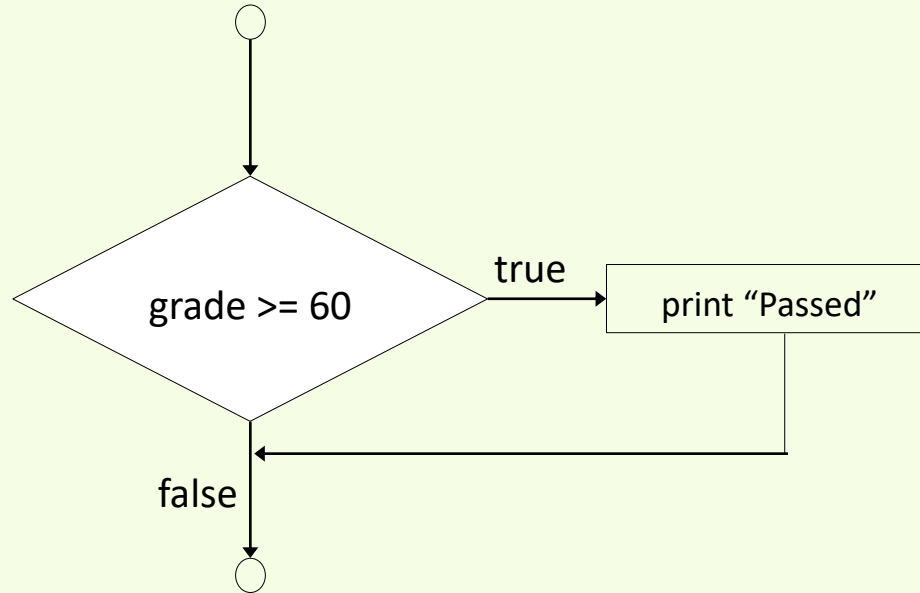
- Selection structure:
 - Used to choose among alternative courses of action
 - Pseudocode: *If student's grade is greater than or equal to 60*
Print "Passed"
- If condition **true**
 - Print statement executed and program goes on to next statement.
 - If **false**, print statement is ignored and the program goes onto the next statement.
 - Indenting makes programs easier to read (使用缩进)
 - C ignores whitespace characters.
- Pseudocode statement in C:

```
if ( grade >= 60 )  
    printf( "Passed\n" );
```



The `if` Selection Structure (II)

- Diamond symbol (decision symbol) - indicates decision is to be made
 - Contains an expression that can be **true** or **false**
 - Test the condition, follow appropriate path
- **if** structure is a single-entry/single-exit structure.



A decision can be made on any expression.
zero - **false**
nonzero - **true**
Example:
(3 - 4) is **true**



The `if/else` Selection Structure (I)

- `if`
 - Only performs an action if the condition is **true**.
- `if/else`
 - A different action when condition is **true** than when condition is **false**

- Psuedocode: *If student's grade is greater than or equal to 60*
 Print "Passed"
 else
 Print "Failed"

– Note spacing/indentation conventions (注意间距/缩进规范)

- C code:

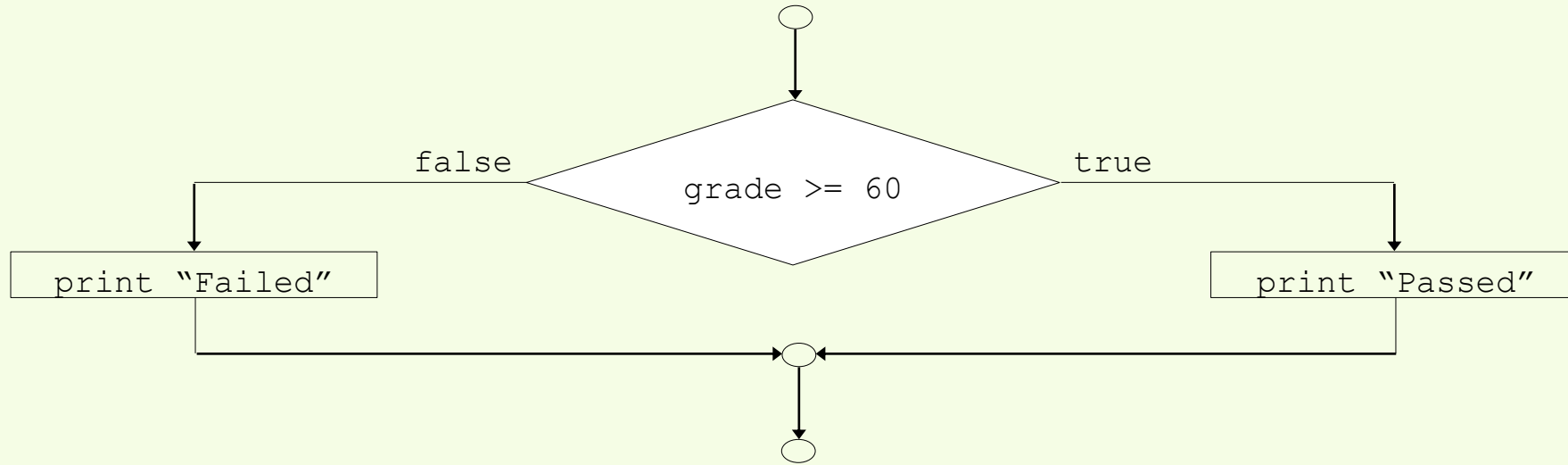
```
if ( grade >= 60 )  
    printf( "Passed\n" );  
else  
    printf( "Failed\n" );
```

or

```
if ( grade >= 60 ){  
    printf( "Passed\n" );  
}  
else{  
    printf( "Failed\n" );  
}
```



The if/else Selection Structure (II)



- Ternary conditional operator (?:)
 - Takes three arguments (condition, value if **true**, value if **false**)
 - Our pseudocode could be written:
`printf( "%s\n", grade >= 60 ? "Passed" : "Failed" );`
OR
`grade >= 60 ? printf( "Passed\n" ) : printf( "Failed\n" );`



The if/else Selection Structure (III)

- Compound statement: (单纯由语句组成的复合语句)
 - Set of statements within a pair of braces
 - Example:

```
if ( grade >= 60 )  
    printf( "Passed.\n" );  
else {  
    printf( "Failed.\n" );  
    printf( "You must take this course again.\n" );  
}
```
 - Without the braces,

```
printf( "You must take this course again.\n" );
```

would be automatically executed
- Block: compound statements with declarations (包含变量声明的复合语句)



The `if/else` Selection Structure (IV)

- Nested `if/else` structures

- Test for multiple cases by placing `if/else` selection structures inside `if/else` selection structures

If student's grade is greater than or equal to 90

Print "A"

else

If student's grade is greater than or equal to 80

Print "B"

else

If student's grade is greater than or equal to 70

Print "C"

else

If student's grade is greater than or equal to 60

Print "D"

else

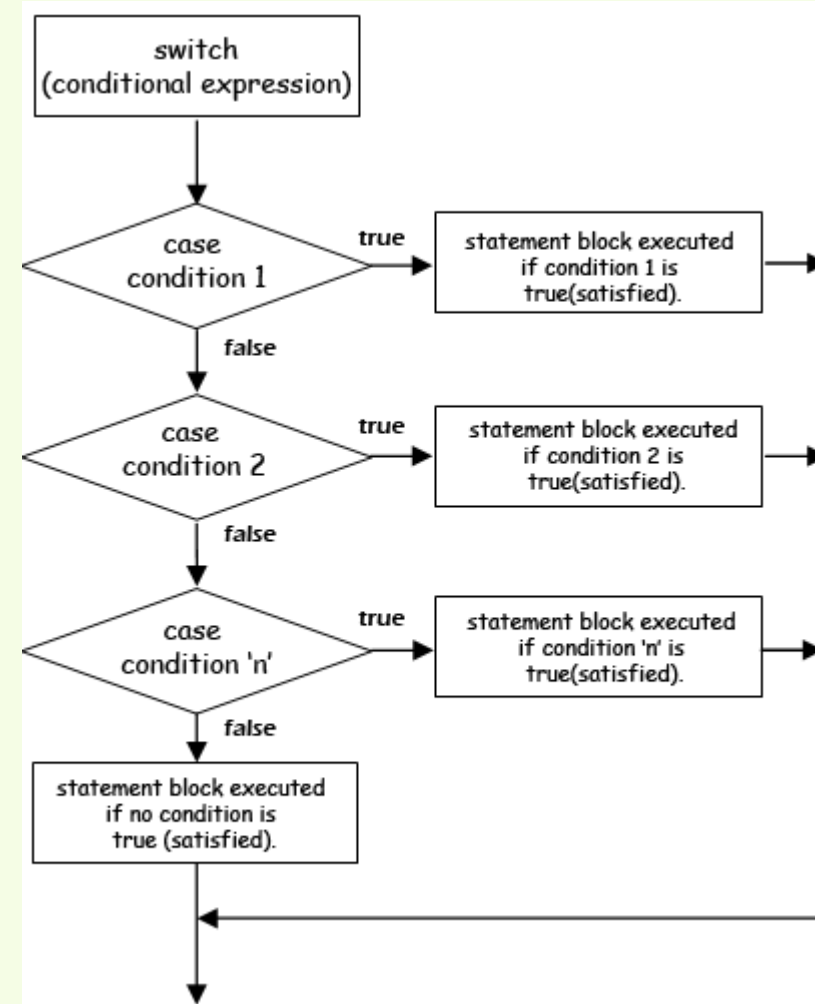
Print "F"

- Once condition is met, rest of statements skipped
- Deep indentation (多层嵌套造成的过度缩进) usually not used in practice



The Switch Selection Structure (I)

- The `switch` statement is a multi-way decision that tests whether an expression matches one of a number of **constant integer** values





The Switch Selection Structure (II)

```
char grade;  
float GPA=0;  
scanf ("%c",&grade) ;  
switch (grade) {  
    case 'A':  
        GPA+=4.0;  
        break;  
    case 'B':  
        GPA+=3.5;  
        break;  
    default:  
        GPA+=2.7;  
}
```

case "A" 可以吗?

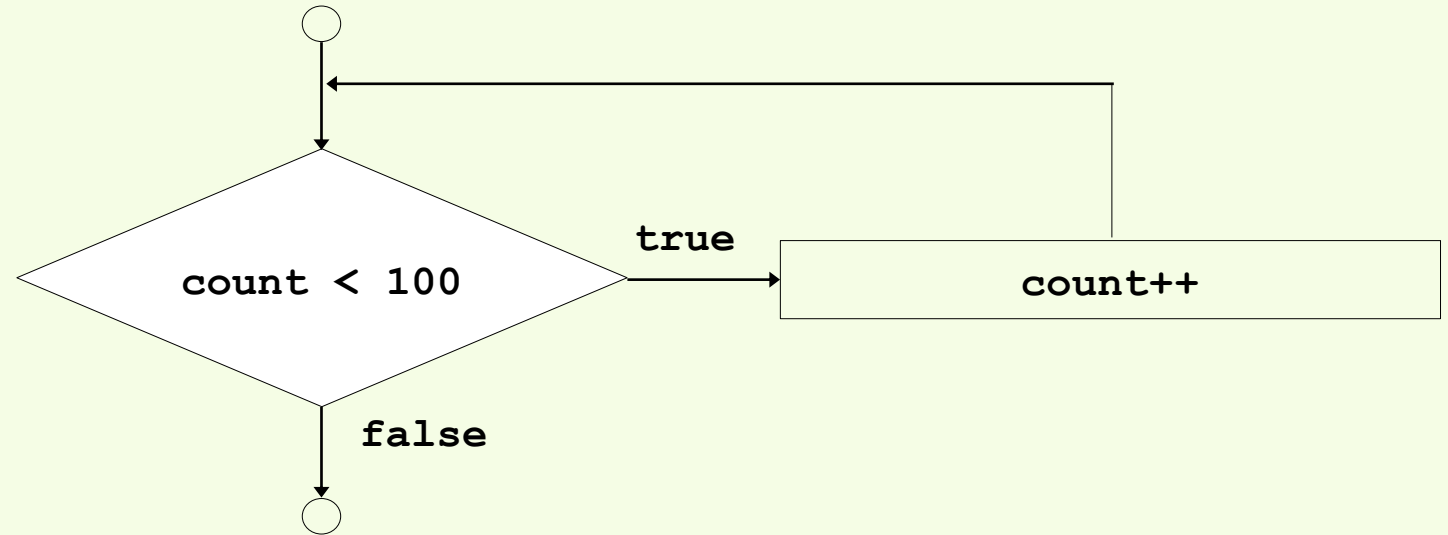
- The `break` statement causes an immediate exit from the switch.



The while Repetition Structure (I)

- Repetition structure
 - Programmer to specify an action to be repeated while some condition remains **true**
- Example:

```
int count = 1;  
while ( count < 100 )  
    count++;
```





The `while` Repetition Structure (II)

- Repetition structure

- Programmer to specify an action to be repeated while some condition remains **true**
- Psuedocode: *While there are more items on my shopping list
Purchase next item and cross it off my list*
- **while** loop repeated until condition becomes **false**

```
int count = 0;  
while ( count < 100 )  
    count++;
```



When stopped,
count =?



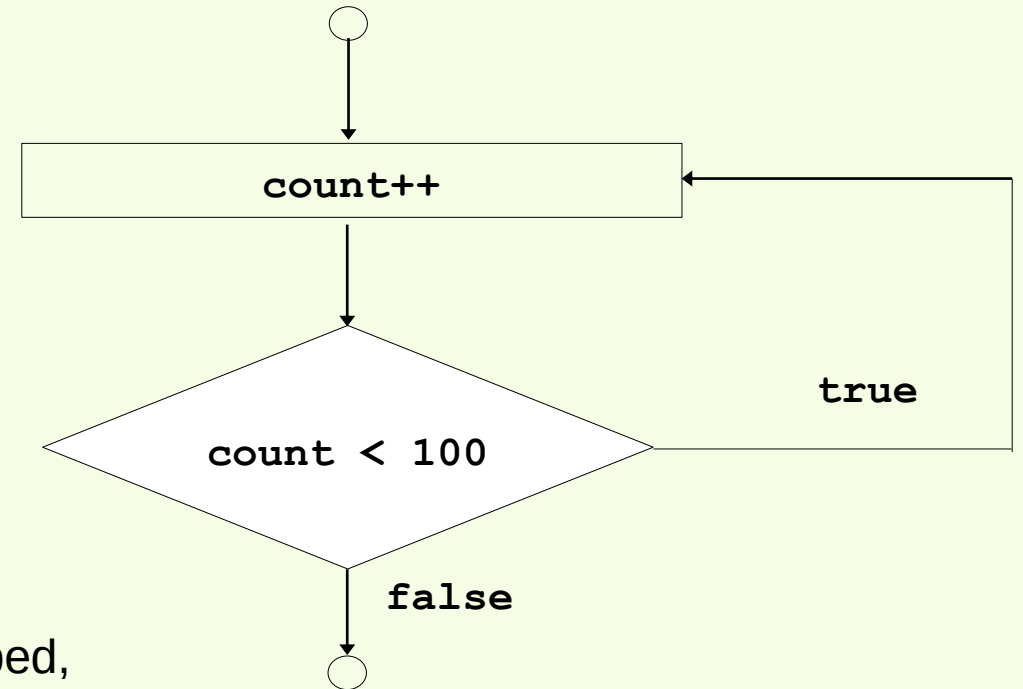
The Do Repetition Structure (I)

- Repetition, statement will be executed once before do the judgement

```
do{  
    statement(s) ;  
  
}while( condition ) ;
```

```
int count=1;  
do {  
    count++;  
} while (count<100);
```

When stopped,
count =?





Exercise 1

- Input 4 3, output? count =?
- Input 3 5, output? count =?
- What does this code do?

```
#include <stdio.h>

int main()
{
    int a,n,count=1;
    long int sn=0, tn=0;
    scanf("%d %d", &a, &n);
    while(count<=n)
    {
        tn=tn+a;
        sn=sn+tn;
        a=a*10;
        count++;
    }
    printf("%ld\n",sn);
}
```



Exercise 2

- Narcissistic number

– For a number n in base $b > 1$, it's has k digitals, We define a narcissistic function $F_b(n)$: $\mathbb{N} \rightarrow \mathbb{N}$

$$F_b(n) = \sum_{i=0}^{k-1} d_i^k$$

– N is a narcissistic number if $F_b(n) = n$

自幂数，特指一个 k 位数其各位数字 k 次幂之和等于自身，其中最常见的是三位数的"水仙花数"

三位的水仙花数

四位的四叶玫瑰数

五位的五角星数

六位的六合数

七位的北斗七星数

八位的八仙数

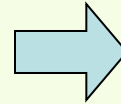
九位的九九重阳数



The “for” loop (shorthand of such while loop)

```
float pow(float x, uint exp)
{
    float result=1.0;
    int i;
    i=0;
    while (i < exp) {
        result = result * x;
        i++;
    }
    return result;
}
```

```
int main(int argc, char **argv)
{
    float p;
    p = pow(10.0, 5);
    printf("p = %f\n", p);
    return 0;
}
```



```
float pow(float x, uint exp)
{
    float result=1.0;
    int i;
    for (i=0; (i < exp); i++) {
        result = result * x;
    }
    return result;
}
```

```
int main(int argc, char **argv)
{
    float p;
    p = pow(10.0, 5);
    printf("p = %f\n", p);
    return 0;
}
```



Josephus

- 这是一个著名的理论数学问题和计算机算法问题，源自古代历史学家弗拉维奥·约瑟夫斯（Flavius Josephus）的传说：
- 约瑟夫斯和40名士兵被罗马军队围困，他们决定宁死不降，于是站成一个圆圈，每数到第3个人就自杀，直到所有人死亡。
- 约瑟夫斯不希望死，于是他快速计算出了自己应该站的位置，从而成为最后幸存者之一。

```
lib.c
src > C lib.c > ...
1  #include "lib.h"
2
3  int solve_std(int n) {
4      // 从剩余人数为 k 时的胜者
5      // (剩余人数 1 时为 0 号) 反推剩余人数为 k + 1 时的胜者。
6      // count 表示模拟情况的剩余人数。
7      int count = 2;
8      // pre_winner 表示当前剩余人数情况下的胜者编号，
9      // 剩余人数为 1 时，胜者为 0 号，也就是初始值。
10     int pre_winner = 0;
11     // 当前要求的答案是剩余 count 个人时，从 0 号开始报数的胜者。
12     while (count <= n) {
13         // 若剩余人数为 k 时胜者为 p 号，
14         // 则剩余人数为 k + 1 时的 (p + 2) 号
15         // 在一次初始报数后就会成为 p 号，因此他将成为胜者。
16         pre_winner = (pre_winner + 2) % count;
17         count += 1;
18     }
19     return pre_winner;
20 }
21
```



Josephus

```
lib.c
src > lib.c > ...
1  #include "lib.h"
2
3  int solve_std(int n) {
4      // 从剩余人数为 k 时的胜者
5      // (剩余人数 1 时为 0 号) 反推剩余人数为 k + 1 时的胜者。
6      // count 表示模拟情况的剩余人数。
7      int count = 2;
8      // pre_winner 表示当前剩余人数情况下的胜者编号,
9      // 剩余人数为 1 时, 胜者为 0 号, 也就是初始值。
10     int pre_winner = 0;
11     // 当前要求的答案是剩余 count 个人时, 从 0 号开始报数的胜者。
12     while (count <= n) {
13         // 若剩余人数为 k 时胜者为 p 号,
14         // 则剩余人数为 k + 1 时的 (p + 2) 号
15         // 在一次初始报数后就会成为 p 号, 因此他将成为胜者。
16         pre_winner = (pre_winner + 2) % count;
17         count += 1;
18     }
19     return pre_winner;
20 }
21
```

```
for (int count=2; count<=n; count++)
    prewinner = (prewinner+2)% count;
```



The for loop structure (II)

- In a for statement, it is possible to place multiple expressions in the various parts

```
for (i = 0, j = strlen(s)-1; i < j; i++, j--)  
    c = s[i], s[i] = s[j], s[j] = c;
```

- `continue` is used to causes the next iteration of the enclosing `for`

```
for (i = 0; i < n; i++)  
    if (a[i] < 0) /* skip negative elements */  
        continue;  
    ... /* do positive elements */
```



Time complexity



Best, Worst, Average Cases

Not all inputs of a given size take the same time to run.

Sequential search for K in an array of n integers:

- Begin at first element in array and look at each element in turn until K is found

Best case: The first element is K

Worst case: The last element is K

Average case: Go halfway through the array



Which Analysis to Use?

While average time appears to be the fairest measure, it may be difficult to determine.

When is the worst case time important?

- Real-time applications e.g. air traffic control system



Faster Computer or Algorithm?

What happens when we buy a computer 10 times faster?

$T(n)$	n	n'	Change	n'/n
$10n$	1,000	10,000	$n' = 10n$	10
$20n$	500	5,000	$n' = 10n$	10
$5n \log n$	250	1,842	$\sqrt{10}n < n' < 10n$	7.37
$2n^2$	70	223	$n' = \sqrt{10}n$	3.16
2^n	13	16	$n' = n + 3$	1.25



Asymptotic Analysis: big-Oh, big-Omega, Theta

Comparaisons asymptotiques

► On a deux fonctions f et g ($g > 0$) définies sur $[a, +\infty[$, on veut les comparer au voisinage de $+\infty$.

- f est **négligeable devant** g (au voisinage de $+\infty$) si :

$$\frac{f(x)}{g(x)} \xrightarrow{x \rightarrow +\infty} 0, \text{ on écrit } f(x) = o_{+\infty}(g(x)).$$

- f est **un grand O de** g si :

$$\exists M > 0, \forall x \in [a, +\infty[, \left| \frac{f(x)}{g(x)} \right| \leq M, \text{ on écrit } f(x) = O_{+\infty}(g(x)).$$

- f est **un grand oméga de** g si :

$$\exists M > 0, \forall x \in [a, +\infty[, \left| \frac{f(x)}{g(x)} \right| \geq M, \text{ on écrit } f(x) = \Omega_{+\infty}(g(x)).$$



Asymptotic Analysis: Big-Oh

- Definition:

- For $T(n)$ a non-negatively valued function, $T(n)$ is in the set $O(f(n))$ if there exist two positive constants c and n_0 such that $T(n) \leq cf(n)$ for all $n > n_0$.

一般情况下，算法中基本操作重复执行的次数是问题规模 n 的某个函数，用 $T(n)$ 表示，若有某个辅助函数 $f(n)$ ，使得当 n 趋近于无穷大时， $T(n)/f(n)$ 的极限值为不等于零的常数，则称 $f(n)$ 是 $T(n)$ 的同数量级函数。记作 $T(n)=O(f(n))$ ，称 $O(f(n))$ 为算法的渐进时间复杂度，简称时间复杂度。

- Usage:

- The algorithm is in $O(n^2)$ in [best, average, worst] case.

- Meaning:

- For all data sets big enough (i.e., $n > n_0$), the algorithm always executes in less than $cf(n)$ steps in [best, average, worst] case.



Big-Oh Notation (cont)

Big-Oh notation indicates an upper bound.

Example: If $T(n) = 3n^2$ then $T(n)$ is in $O(n^2)$.

时间复杂度常用大O符号表述，不包括这个函数的低阶项和首项系数。

在各种不同算法中，若算法中语句执行次数为一个常数，则时间复杂度为 $O(1)$ 。另外，在时间频度不相同，时间复杂度有可能相同，如 $T(n)=n^2+3n+4$ 与 $T(n)=4n^2+2n+1$ 它们的频度不同，但时间复杂度相同，都为 $O(n^2)$ 。

Wish tightest upper bound:

While $T(n) = 3n^2$ is in $O(n^3)$, we prefer $O(n^2)$.

虽然 $O(n^3)$ 也是它的上界，但应该选择更紧的上界 $O(n^2)$ ，因为它更准确地描述了算法的增长趋势。在分析算法时，我们总是追求最紧的上界（即最小的大O表示），以避免过高估计算法的复杂度。



Big-Oh Examples

Example 1: Finding value X in an array (average cost).



Big-Oh Examples

Example 2: $T(n) = c_1n^2 + c_2n$ in average case.

Example 3: $T(n) = c$.



A Common Misunderstanding

“The best case for my algorithm is $n=1$ because that is the fastest.”

WRONG!

Big-oh refers to a growth rate as n grows to ∞ .

Best case is defined as which input of size n is cheapest among all inputs of size n .



Asymptotic Analysis: Big-Omega

- Definition:
 - For $T(n)$ a non-negatively valued function, $T(n)$ is in the set $\Omega(g(n))$ if there exist two positive constants c and n_0 such that $T(n) \geq cg(n)$ for all $n > n_0$.
- Meaning:
 - For all data sets big enough (i.e., $n > n_0$), the algorithm always executes in more than $cg(n)$ steps.

Lower bound.



Big-Omega Example

$$T(n) = c_1 n^2 + c_2 n.$$



Theta Notation

When big-Oh and big-Omega meet, we indicate this by using Θ (big-Theta) notation.

- Definition:
 - An algorithm is said to be $\Theta(h(n))$ if it is in $O(h(n))$ and it is in $\Omega(h(n))$.



A Common Misunderstanding

Confusing worst case with upper bound.

Upper bound refers to a growth rate.

Worst case refers to the worst input from among the choices for possible inputs of a given size.



Simplifying Rules

1. If $f(n)$ is in $O(g(n))$ and $g(n)$ is in $O(h(n))$, then $f(n)$ is in $O(h(n))$.
2. If $f(n)$ is in $O(kg(n))$ for any constant $k > 0$, then $f(n)$ is in $O(g(n))$.
3. If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$, then $(f_1 + f_2)(n)$ is in $O(\max(g_1(n), g_2(n)))$.
4. If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$ then $f_1(n)f_2(n)$ is in $O(g_1(n)g_2(n))$.



Running Time Examples (1)

Example 1:

```
a = b;
```

Example 2:

```
sum = 0;  
for (i=1; i<=n; i++)  
    sum += n;
```



Running Time Examples (2)

Example 3:

```
sum = 0;
for (i=1; i<=n; j++)
    for (j=1; j<=i; i++)
        sum++;
for (k=0; k<n; k++)
    A[k] = k;
```



Running Time Examples (3)

Example 4:

```
sum1 = 0;  
for (i=1; i<=n; i++)  
    for (j=1; j<=n; j++)  
        sum1++;
```

```
sum2 = 0;  
for (i=1; i<=n; i++)  
    for (j=1; j<=i; j++)  
        sum2++;
```



Running Time Examples (4)

Example 5:

```
sum1 = 0;  
for (k=1; k<=n; k*=2)  
    for (j=1; j<=n; j++)  
        sum1++;
```

```
sum2 = 0;  
for (k=1; k<=n; k*=2)  
    for (j=1; j<=k; j++)  
        sum2++;
```